DATA SCIENCE LABORATORY
PRACTICAL RECORD

Python Programs • Expected Outputs • Visualizations • Results

Name: Nilavazhagan Register No.: 250701483

Class / Section: CSE K

Experiment Index

1.a — Trend of Data Science Job Postings

1.b — Distribution of Data Science Roles

1.c — Structured, Semi-structured and Unstructured Data

1.d — Encrypt and Decrypt Sensitive Data using Fernet

2 — CSV Data Analysis, Preprocessing and Visualization

3.a — Data Visualization using Line Plot

3.b — Data Visualization using Bar Chart

3.c — Data Visualization using Pie Chart

4 — Count Word Frequency in a Body of Text

5 — Data Collection and Initial Exploration

6 — Time Series Analysis of Website Traffic

7 — Random Sampling and Sampling Distribution

8 — Conduct Z-Test using Python

9 — Matplotlib Library and its Uses

10 — Array Functions in Data Science (NumPy)

11 — Pandas Library in Data Science

12 — Detect Outliers in a Dataset using IQR

13 — Handle Missing Values and Inappropriate Data

14 — Feature Scaling

15 — Data Preprocessing in Data Science

16 — EDA: Quantitative and Qualitative Analysis

17 — Linear Regression

18 — K-Nearest Neighbors (KNN) Algorithm

19 — Logistic Regression

20 — K-Means Clustering

1.a — Trend of Data Science Job Postings

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

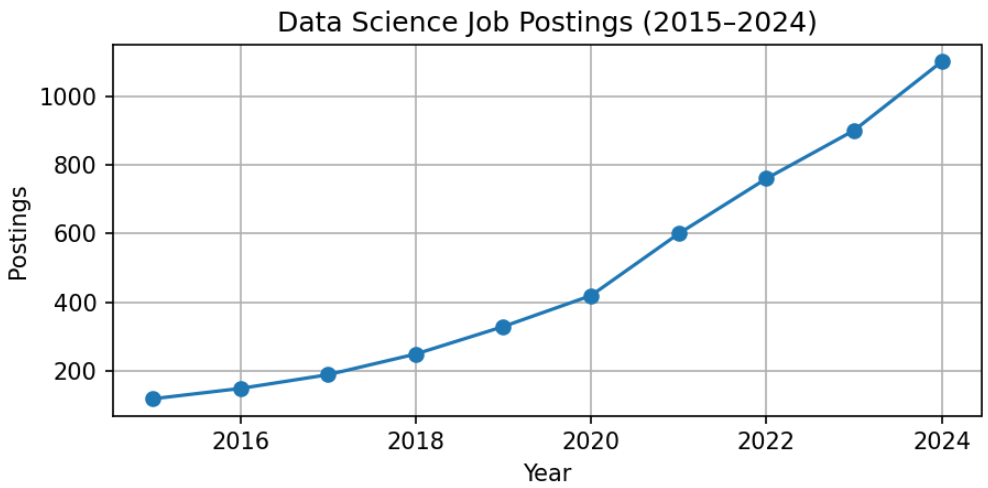
```
import pandas as pd
import matplotlib.pyplot as plt
data = {
"Year": [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024],
"Job_Postings": [120, 150, 190, 250, 330, 420, 600, 760, 900, 1100]
}
df = pd.DataFrame(data)
print(df)
print("\nPercentage growth from 2015 to 2024:",
round((df.Job_Postings.iloc[-1] - df.Job_Postings.iloc[0]) /
df.Job_Postings.iloc[0] * 100, 2), "%")
plt.plot(df.Year, df.Job_Postings, marker="o")
plt.title("Data Science Job Postings (2015–2024)")
plt.xlabel("Year"); plt.ylabel("Number of postings")
plt.grid(True); plt.tight_layout(); plt.show()
```

Expected Output

```
Year Job_Postings
2015 120
2016 150
2017 190
```

```
2018 250
2019 330
2020 420
2021 600
2022 760
2023 900
2024 1100
Percentage growth from 2015 to 2024: 816.67 %
```

Visualization



Result

Line graph shows a strong upward trend in data science job postings over the decade.

1.b — Distribution of Data Science Roles

Aim

To perform the experiment using Python and interpret the generated output.

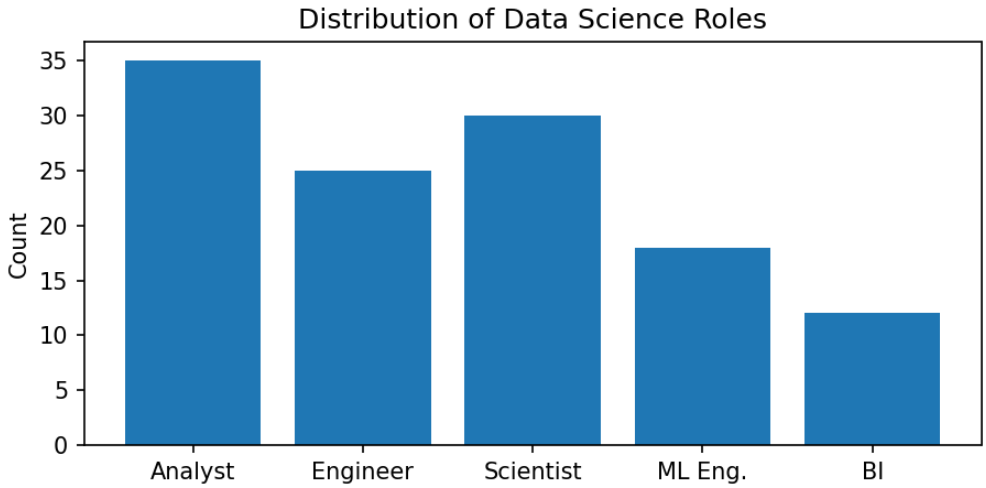
Program / Input Code

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({
"Role": ["Data Analyst", "Data Engineer", "Data Scientist",
"ML Engineer", "BI Analyst"],
"Count": [35, 25, 30, 18, 12]
})
print(df)
print("\nMost common role:", df.loc[df.Count.idxmax(), "Role"])
plt.bar(df.Role, df.Count)
plt.title("Distribution of Data Science Roles")
plt.xlabel("Role"); plt.ylabel("Number of job postings")
plt.xticks(rotation=25); plt.tight_layout(); plt.show()
```

Expected Output

```
Role Count
0 Data Analyst 35
1 Data Engineer 25
2 Data Scientist 30
3 ML Engineer 18
4 BI Analyst 12
Most common role: Data Analyst
```

Visualization



Result

Data Analyst is the most frequent role in this sample dataset.

1.c — Structured, Semi-structured and Unstructured Data

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
import pandas as pd
import json
# Structured: rows and columns
structured = pd.DataFrame({
"ID": [1, 2, 3], "Name": ["Asha", "Bala", "Charan"],
"Mark": [85, 78, 92]
})
print("STRUCTURED DATA")
print(structured)
# Semi-structured: JSON key-value records
semi_structured = [
{"id": 1, "name": "Asha", "skills": ["Python", "SQL"]},
{"id": 2, "name": "Bala", "skills": ["Java"]}
]
print("\nSEMI-STRUCTURED DATA")
print(json.dumps(semi_structured, indent=2))
# Unstructured: free text (no fixed tabular schema)
unstructured = ["Python is easy to learn.", "Data science uses statistics."]
print("\nUNSTRUCTURED DATA")
for sentence in unstructured:
print("-", sentence)
```

Expected Output

```
STRUCTURED DATA
ID Name Mark
0 1 Asha 85
1 2 Bala 78
2 3 Charan 92
SEMI-STRUCTURED DATA
[
{"id": 1, "name": "Asha", "skills": ["Python", "SQL"]},
{"id": 2, "name": "Bala", "skills": ["Java"]}
]
UNSTRUCTURED DATA
- Python is easy to learn.
- Data science uses statistics.
```

Result

Tabular records are structured, JSON records are semi-structured, and free text is unstructured data.

1.d — Encrypt and Decrypt Sensitive Data using Fernet

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
# Install once: pip install cryptography
from cryptography.fernet import Fernet
key = Fernet.generate_key() # Generate a secret key
cipher = Fernet(key)
message = b"Confidential student information"
encrypted = cipher.encrypt(message)
decrypted = cipher.decrypt(encrypted)
print("Original message:", message.decode())
print("Encrypted message:", encrypted.decode())
```

```
print("Decrypted message:", decrypted.decode())
```

Expected Output

```
Original message: Confidential student information
Encrypted message: gAAAAA... (a new encrypted token is generated each run)
Decrypted message: Confidential student information
```

Result

Fernet successfully encrypts the original message and decrypts it back using the same secret key. Keep the key secure.

2 — CSV Data Analysis, Preprocessing and Visualization

Aim

To perform the experiment using Python and interpret the generated output.

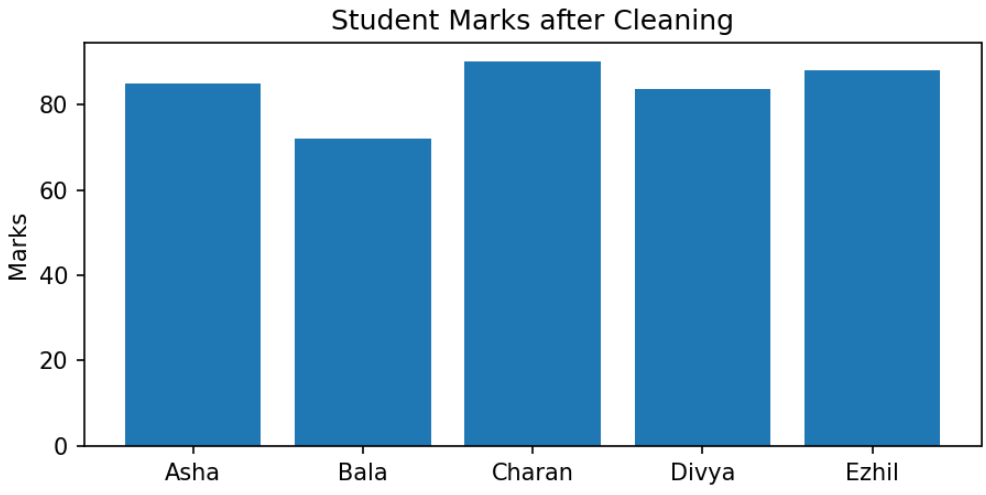
Program / Input Code

```
import pandas as pd
import matplotlib.pyplot as plt
# Create a sample CSV file
sample = pd.DataFrame({
"Name": ["Asha", "Bala", "Charan", "Divya", "Ezhil"],
"Age": [20, 21, None, 22, 20],
"Marks": [85, 72, 90, None, 88]
})
sample.to_csv("students.csv", index=False)
df = pd.read_csv("students.csv")
print("First five rows:\n", df.head())
print("\nMissing values:\n", df.isnull().sum())
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
print("\nCleaned data:\n", df)
print("\nSummary:\n", df.describe())
plt.bar(df.Name, df.Marks)
plt.title("Student Marks"); plt.xlabel("Student"); plt.ylabel("Marks")
plt.tight_layout(); plt.show()
```

Expected Output

```
Missing values:
Name 0
Age 1
Marks 1
Cleaned data:
Name Age Marks
0 Asha 20.0 85.0
1 Bala 21.0 72.0
2 Charan 20.5 90.0
3 Divya 22.0 83.75
4 Ezhil 20.0 88.0
```

Visualization



Result

CSV data was loaded, missing values were filled, summary statistics were generated and marks were visualized.

3.a — Data Visualization using Line Plot

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

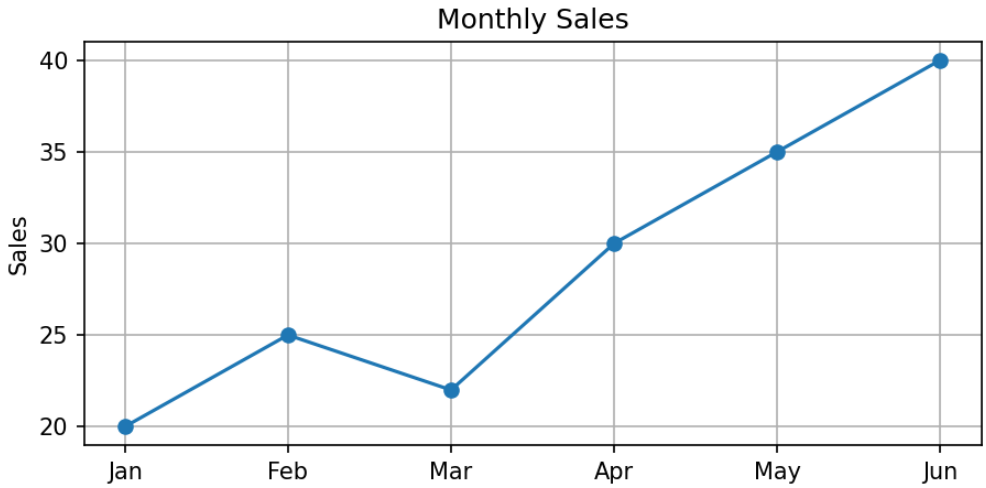
```
import matplotlib.pyplot as plt
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [20, 25, 22, 30, 35, 40]
plt.plot(months, sales, marker="o", linestyle="-", label="Sales")
```

```
plt.title("Monthly Sales")
plt.xlabel("Month"); plt.ylabel("Sales (units)")
plt.legend(); plt.grid(True); plt.tight_layout(); plt.show()
print("Highest sales:", max(sales), "units")
```

Expected Output

```
Highest sales: 40 units
```

Visualization



Result

Line plot displays the change in sales over time; sales reach a maximum of 40 units in June.

3.b — Data Visualization using Bar Chart

Aim

To perform the experiment using Python and interpret the generated output.

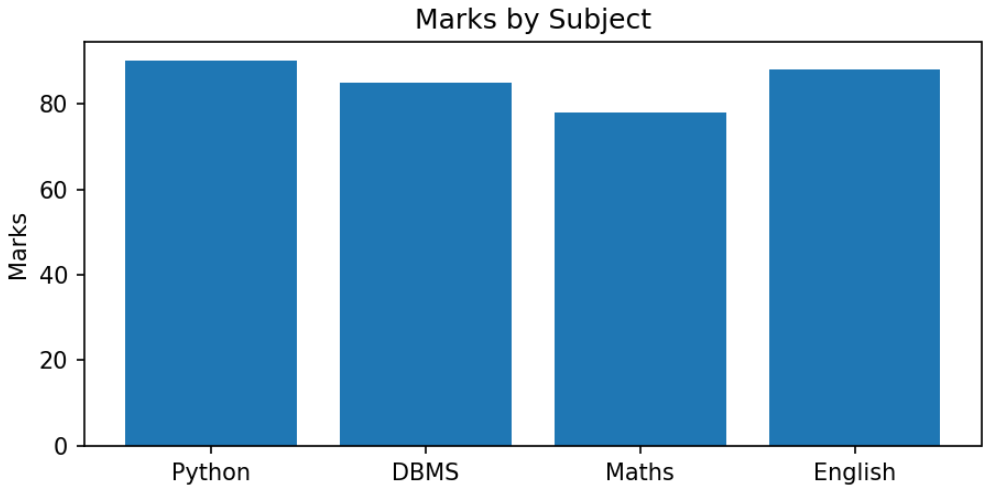
Program / Input Code

```
import matplotlib.pyplot as plt
subjects = ["Python", "DBMS", "Maths", "English"]
marks = [90, 85, 78, 88]
plt.bar(subjects, marks)
plt.title("Marks in Different Subjects")
plt.xlabel("Subject"); plt.ylabel("Marks")
plt.ylim(0, 100); plt.tight_layout(); plt.show()
print("Average mark:", sum(marks) / len(marks))
```

Expected Output

```
Average mark: 85.25
```

Visualization



Result

Bar chart compares marks across subjects. Python has the highest mark (90).

3.c — Data Visualization using Pie Chart

Aim

To perform the experiment using Python and interpret the generated output.

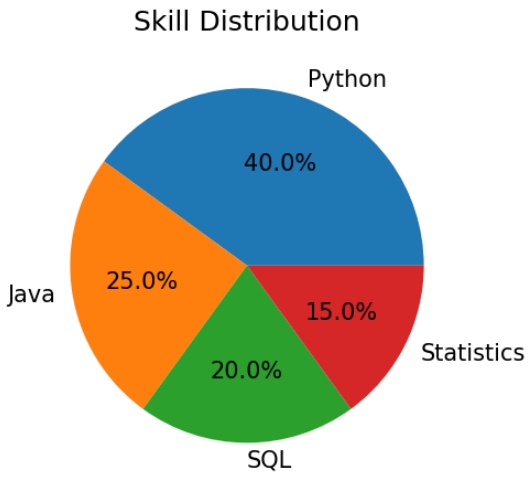
Program / Input Code

```
import matplotlib.pyplot as plt
labels = ["Python", "Java", "SQL", "Statistics"]
values = [40, 25, 20, 15]
plt.pie(values, labels=labels, autopct="%1.1f%%", startangle=90)
plt.title("Skill Distribution")
plt.axis("equal"); plt.show()
print("Total percentage:", sum(values), "%")
```

Expected Output

```
Total percentage: 100 %
```

Visualization



Result

Pie chart shows each skill's share; Python has the largest share at 40%.

4 — Count Word Frequency in a Body of Text

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
import re
from collections import Counter
text = "Data science is useful. Data science uses Python, and Python is powerful."
words = re.findall(r"\b[a-zA-Z]+\b", text.lower())
frequency = Counter(words)
print("Word frequencies:")
for word, count in frequency.most_common():
print(word, ":", count)
```

Expected Output

```
Word frequencies:
data : 2
science : 2
is : 2
python : 2
useful : 1
uses : 1
and : 1
powerful : 1
```

Result

Word frequency counting identifies how many times each word occurs in the given text.

5 — Data Collection and Initial Exploration

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
import pandas as pd
data = {
"Name": ["Asha", "Bala", "Charan", "Divya", "Ezhil"],
"Age": [20, 21, 19, 22, 20],
"Department": ["CSE", "IT", "CSE", "ECE", "IT"],
"Score": [85, 78, 92, 88, 80]
}
```

```
df = pd.DataFrame(data)
print("Dataset:\n", df)
print("\nShape:", df.shape)
print("\nColumns:", list(df.columns))
print("\nData types:\n", df.dtypes)
print("\nInformation:")
df.info()
print("\nStatistical summary:\n", df.describe(include="all"))
print("\nMissing values:\n", df.isnull().sum())
```

Expected Output

```
Shape: (5, 4)
Columns: ['Name', 'Age', 'Department', 'Score']
Missing values: 0 in every column
The describe() output gives count, mean, standard deviation, minimum and maximum for numeric columns.
```

Result

Sample data was collected into a DataFrame and its structure, data types, summary statistics and missing values were examined.

6 — Time Series Analysis of Website Traffic

Aim

To perform the experiment using Python and interpret the generated output.

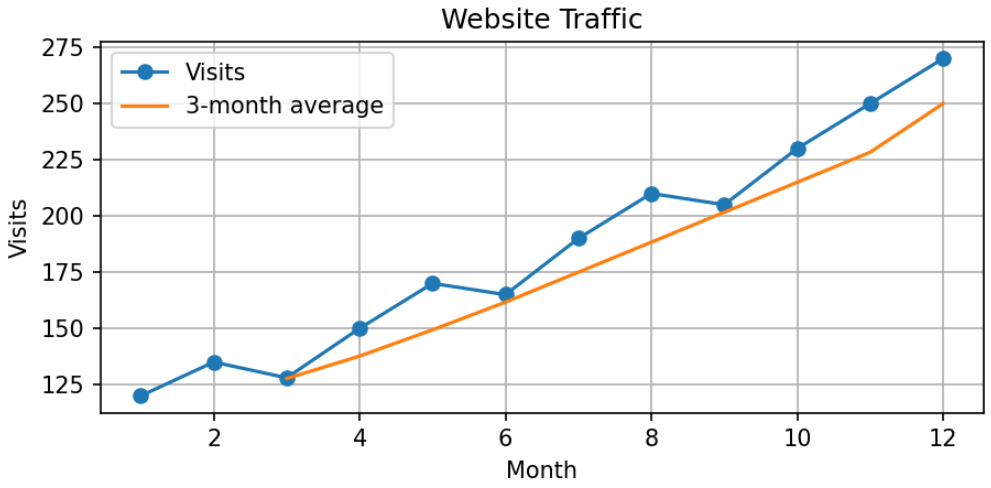
Program / Input Code

```
import pandas as pd
import matplotlib.pyplot as plt
dates = pd.date_range("2024-01-01", periods=12, freq="MS")
visits = [120, 135, 128, 150, 170, 165, 190, 210, 205, 230, 250, 270]
df = pd.DataFrame({"Date": dates, "Visits": visits}).set_index("Date")
df["3_Month_Moving_Average"] = df["Visits"].rolling(3).mean()
print(df)
print("\nTotal visits:", df.Visits.sum())
df[["Visits", "3_Month_Moving_Average"]].plot(marker="o")
plt.title("Website Traffic and 3-Month Moving Average")
plt.xlabel("Month"); plt.ylabel("Visits")
plt.tight_layout(); plt.show()
```

Expected Output

```
Total visits: 2223
The moving average smooths short-term fluctuations and helps reveal the overall trend.
```

Visualization



Result

Website visits generally increase over the year. The moving average makes the upward trend easier to observe.

7 — Random Sampling and Sampling Distribution

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

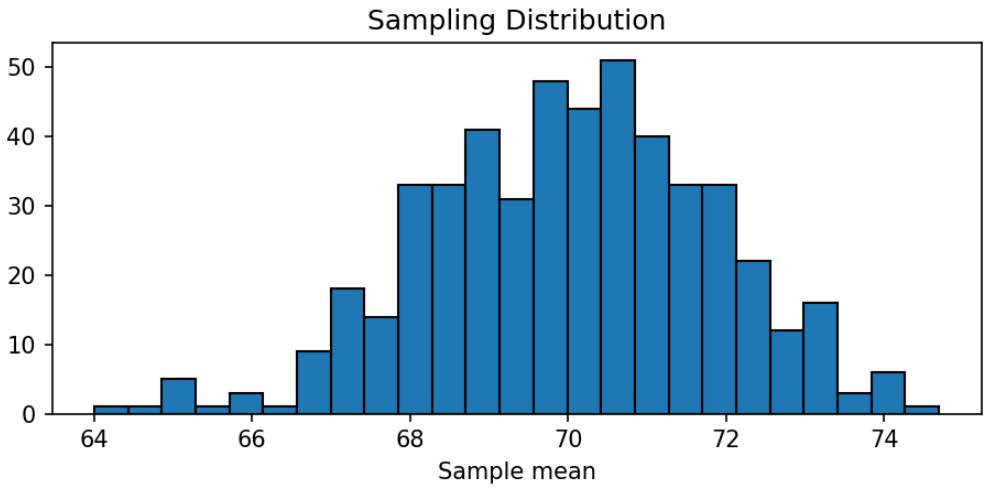
```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
population = np.random.normal(loc=70, scale=10, size=10000)
sample_means = []
for _ in range(500):
sample = np.random.choice(population, size=30, replace=True)
sample_means.append(sample.mean())
print("Population mean:", round(population.mean(), 2))
print("Mean of sample means:", round(np.mean(sample_means), 2))
print("Standard error of sample means:", round(np.std(sample_means, ddof=1), 2))
```

```
plt.hist(sample_means, bins=25, edgecolor="black")
plt.title("Sampling Distribution of Sample Mean")
plt.xlabel("Sample mean"); plt.ylabel("Frequency")
plt.tight_layout(); plt.show()
```

Expected Output

```
Population mean: approximately 70
Mean of sample means: approximately 70
The histogram of sample means is approximately bell-shaped.
```

Visualization



Result

Repeated random samples produce a sampling distribution centered near the population mean.

8 — Conduct Z-Test using Python

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
from statsmodels.stats.weightstats import ztest
# Example: test whether the sample mean differs from 50
sample = [52, 49, 51, 53, 48, 50, 54, 52, 51, 49,
53, 52, 50, 48, 51, 52, 54, 49, 50, 53]
z_stat, p_value = ztest(sample, value=50)
print("Sample mean:", sum(sample) / len(sample))
print("Z statistic:", round(z_stat, 4))
print("P-value:", round(p_value, 4))
alpha = 0.05
if p_value < alpha:
print("Reject H0: significant difference from 50.")
else:
print("Fail to reject H0: insufficient evidence of a difference.")
```

Expected Output

```
Sample mean: 51.05
Z statistic and p-value are calculated by statsmodels.
Decision is based on comparing p-value with 0.05.
```

Result

The Z-test compares the sample mean with the hypothesized population mean. A p-value below 0.05 indicates statistical significance. This example assumes a known/large-sample standard-error setting.

9 — Matplotlib Library and its Uses

Aim

To perform the experiment using Python and interpret the generated output.

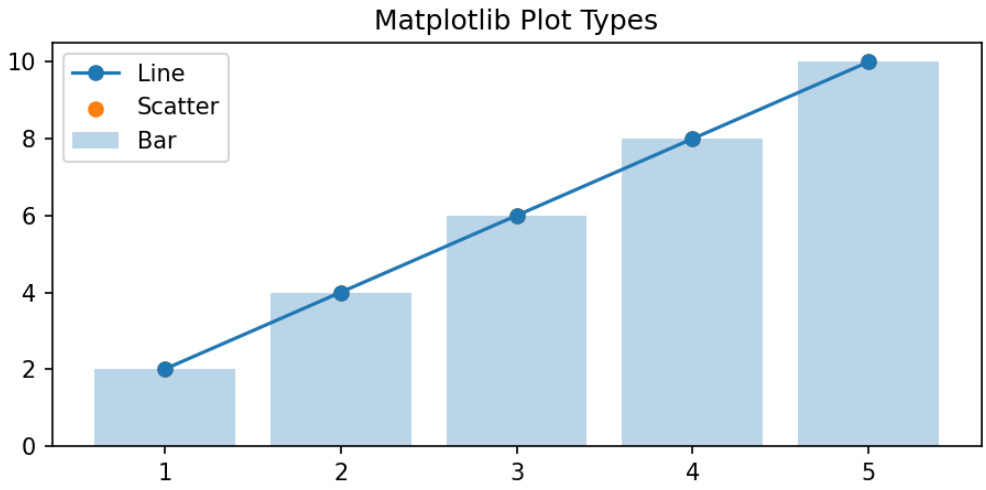
Program / Input Code

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.figure(figsize=(9, 3))
plt.subplot(1, 3, 1)
plt.plot(x, y, marker="o"); plt.title("Line")
plt.subplot(1, 3, 2)
plt.bar(x, y); plt.title("Bar")
plt.subplot(1, 3, 3)
plt.scatter(x, y); plt.title("Scatter")
plt.tight_layout(); plt.show()
print("Matplotlib supports line, bar, scatter, histogram and many other plots.")
```

Expected Output

```
Matplotlib supports line, bar, scatter, histogram and many other plots.
```

Visualization



Result

Matplotlib was used to create line, bar and scatter plots for data visualization.

10 — Array Functions in Data Science (NumPy)

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
print("Array:", a)
print("Shape:", a.shape)
print("Mean:", np.mean(a))
print("Median:", np.median(a))
print("Sum:", np.sum(a))
print("Maximum:", np.max(a))
print("Minimum:", np.min(a))
print("Reshaped array:\n", a.reshape(5, 1))
print("Array + 5:", a + 5)
```

Expected Output

```
Array: [10 20 30 40 50]
Shape: (5,)
Mean: 30.0
Median: 30.0
Sum: 150
Maximum: 50
Minimum: 10
Reshaped array:
[[10]
[20]
[30]
[40]
[50]]
Array + 5: [15 25 35 45 55]
```

Result

NumPy array operations perform numerical calculations efficiently and support reshaping and vectorized arithmetic.

11 — Pandas Library in Data Science

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
import pandas as pd
df = pd.DataFrame({
"Name": ["Asha", "Bala", "Charan"],
"Age": [20, 21, 19],
"Marks": [85, 78, 92]
})
print("DataFrame:\n", df)
print("\nFirst two rows:\n", df.head(2))
print("\nMarks column:\n", df["Marks"])
print("\nAverage marks:", df["Marks"].mean())
print("\nSorted by marks:\n", df.sort_values("Marks", ascending=False))
print("\nShape:", df.shape)
```

Expected Output

```
Average marks: 85.0
Shape: (3, 3)
The DataFrame displays three student records with columns Name, Age and Marks.
```

Result

Pandas was used to create, select, summarize and sort tabular data.

12 — Detect Outliers in a Dataset using IQR

Aim

To perform the experiment using Python and interpret the generated output.

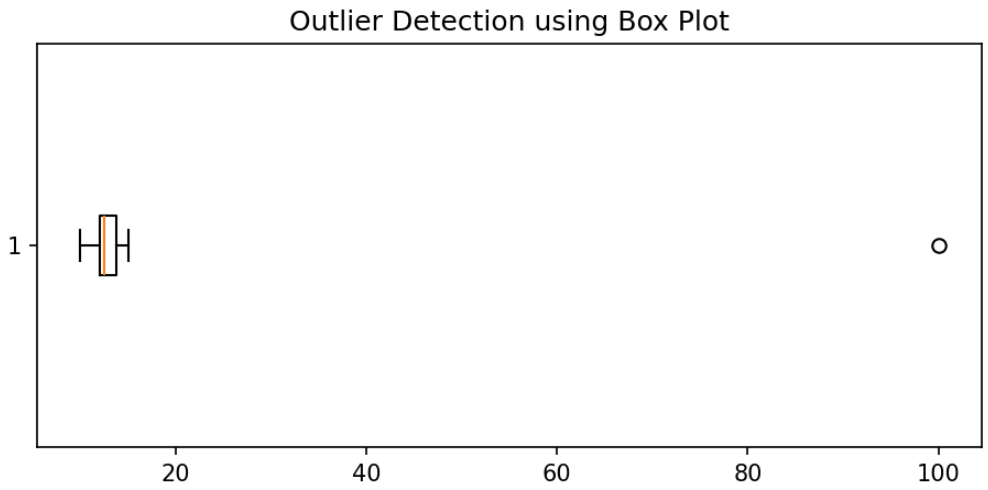
Program / Input Code

```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.Series([10, 12, 12, 13, 12, 14, 15, 13, 12, 100])
Q1 = data.quantile(0.25)
Q3 = data.quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = data[(data < lower) | (data > upper)]
print("Q1:", Q1, "Q3:", Q3, "IQR:", IQR)
print("Lower bound:", lower, "Upper bound:", upper)
print("Outliers:", outliers.tolist())
plt.boxplot(data, vert=False)
plt.title("Box Plot for Outlier Detection")
plt.xlabel("Value"); plt.show()
```

Expected Output

```
Q1, Q3 and IQR are calculated from the sample.
Outliers: [100]
```

Visualization



Result

The IQR method identifies 100 as an outlier in the sample dataset.

13 — Handle Missing Values and Inappropriate Data

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
"Age": [20, 21, np.nan, 200, 22],
"Marks": [85, np.nan, 90, 75, 88],
"Name": ["Asha", "Bala", "Charan", "Divya", "Ezhil"]
})
print("Before cleaning:\n", df)
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
# Treat impossible age 200 as missing, then impute
df.loc[df["Age"] > 100, "Age"] = np.nan
df["Age"] = df["Age"].fillna(df["Age"].median())
print("\nAfter cleaning:\n", df)
print("\nMissing values:\n", df.isnull().sum())
```

Expected Output

```
Before cleaning: dataset contains missing values and an invalid age of 200.
```

```
After cleaning: missing values are imputed and invalid age is replaced.
Missing values: 0 in all columns.
```

Result

Missing and inappropriate values were detected and handled, producing a clean dataset.

14 — Feature Scaling

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
df = pd.DataFrame({"Age": [20, 30, 40, 50],
"Salary": [20000, 40000, 60000, 80000]})
print("Original data:\n", df)
minmax = MinMaxScaler()
standard = StandardScaler()
print("\nMin-Max scaled:\n",
pd.DataFrame(minmax.fit_transform(df), columns=df.columns).round(2))
print("\nStandard scaled:\n",
pd.DataFrame(standard.fit_transform(df), columns=df.columns).round(2))
```

Expected Output

```
Min-Max scaled:
Age Salary
0 0.00 0.00
1 0.33 0.33
2 0.67 0.67
3 1.00 1.00
Standard scaled values are centered around 0 with unit variance.
```

Result

Min-Max scaling transforms features to the 0–1 range, while standardization transforms them to approximately zero mean and unit variance.

15 — Data Preprocessing in Data Science

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
df = pd.DataFrame({
"Age": [20, 25, None, 30],
"City": ["Chennai", "Madurai", "Chennai", "Trichy"],
"Income": [20000, 30000, 25000, 40000]
})
print("Raw data:\n", df)
df["Age"] = df["Age"].fillna(df["Age"].median())
encoder = LabelEncoder()
df["City_Code"] = encoder.fit_transform(df["City"])
scaler = MinMaxScaler()
df[["Age_Scaled", "Income_Scaled"]] = scaler.fit_transform(df[["Age", "Income"]])
print("\nPreprocessed data:\n", df)
```

Expected Output

```
Raw data contains one missing Age value and a categorical City column.
Preprocessed data contains imputed age, encoded city and scaled numeric features.
```

Result

Data preprocessing handled missing values, encoded categorical data and scaled numerical features.

16 — EDA: Quantitative and Qualitative Analysis

Aim

To perform the experiment using Python and interpret the generated output.

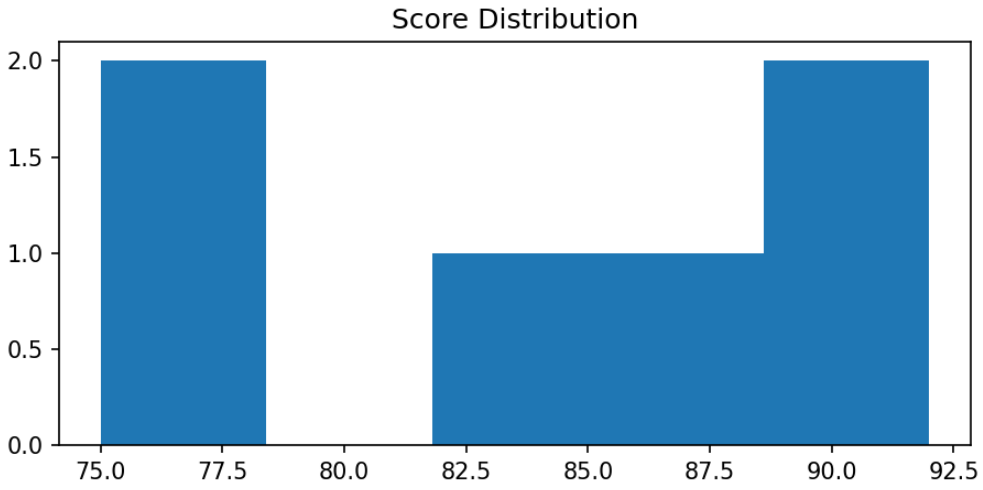
Program / Input Code

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({
"Department": ["CSE", "IT", "CSE", "ECE", "IT", "CSE"],
"Score": [85, 78, 92, 75, 88, 90],
"Age": [20, 21, 19, 20, 22, 21]
})
print("QUANTITATIVE ANALYSIS")
print(df[["Score", "Age"]].describe())
print("\nQUALITATIVE ANALYSIS")
print(df["Department"].value_counts())
fig, axes = plt.subplots(1, 2, figsize=(8, 3))
df["Score"].hist(ax=axes[0], bins=5)
axes[0].set_title("Score Distribution")
df["Department"].value_counts().plot(kind="bar", ax=axes[1])
axes[1].set_title("Department Frequency")
plt.tight_layout(); plt.show()
```

Expected Output

```
Quantitative analysis: describe() reports count, mean, standard deviation, min, quartiles and max.
Qualitative analysis: value_counts() reports the frequency of each department.
```

Visualization



Result

Quantitative variables were summarized statistically and qualitative categories were examined using frequency counts.

17 — Linear Regression

Aim

To perform the experiment using Python and interpret the generated output.

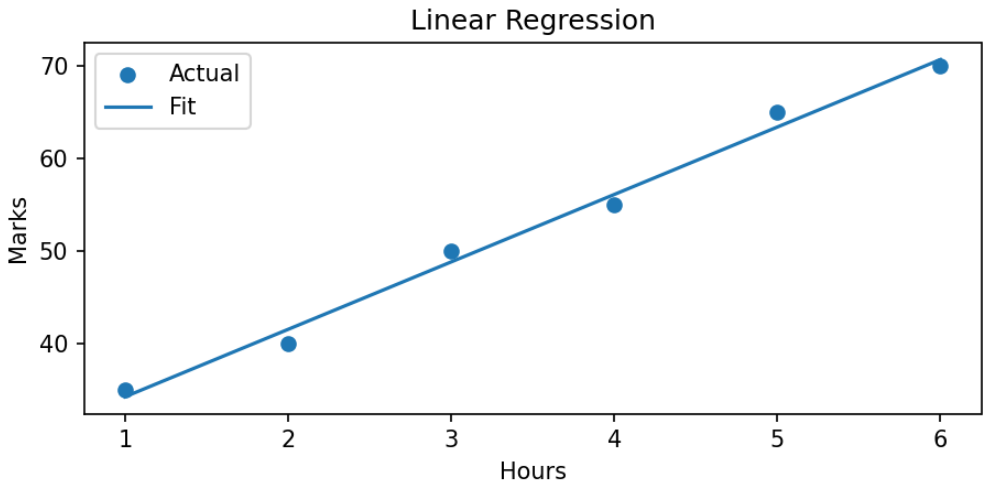
Program / Input Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# Hours studied and exam marks
X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
y = np.array([35, 40, 50, 55, 65, 70])
model = LinearRegression()
model.fit(X, y)
pred = model.predict(X)
print("Slope:", round(model.coef_[0], 2))
print("Intercept:", round(model.intercept_, 2))
print("R2 score:", round(r2_score(y, pred), 3))
print("Predicted mark for 7 hours:", round(model.predict([[7]])[0], 2))
plt.scatter(X, y, label="Actual")
plt.plot(X, pred, label="Regression line")
plt.xlabel("Hours studied"); plt.ylabel("Marks")
plt.title("Linear Regression"); plt.legend(); plt.show()
```

Expected Output

```
The model learns a positive relationship between study hours and marks.
Predicted mark for 7 hours: approximately 80 (depends on fitted line).
```

Visualization



Result

Linear regression fits a straight line to predict a continuous target variable (marks) from study hours.

18 — K-Nearest Neighbors (KNN) Algorithm

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
iris.data, iris.target, test_size=0.2, random_state=42, stratify=iris.target)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, pred), 3))
print("\nClassification report:\n",
classification_report(y_test, pred, target_names=iris.target_names))
```

Expected Output

```
Accuracy: typically around 0.93–1.00 for this split.
Classification report displays precision, recall, f1-score and support for each Iris class.
```

Result

KNN classifies a new sample using the majority class among its five nearest training samples.

19 — Logistic Regression

Aim

To perform the experiment using Python and interpret the generated output.

Program / Input Code

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
data.data, data.target, test_size=0.2, random_state=42, stratify=data.target)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, pred), 3))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print("\n", classification_report(y_test, pred, target_names=data.target_names))
```

Expected Output

```
Accuracy: commonly around 0.96–0.99 for this dataset and split.
Confusion matrix and classification report show the classification performance.
```

Result

Logistic regression predicts a categorical class using probabilities. The model is evaluated using accuracy, confusion matrix, precision and recall.

20 — K-Means Clustering

Aim

To perform the experiment using Python and interpret the generated output.

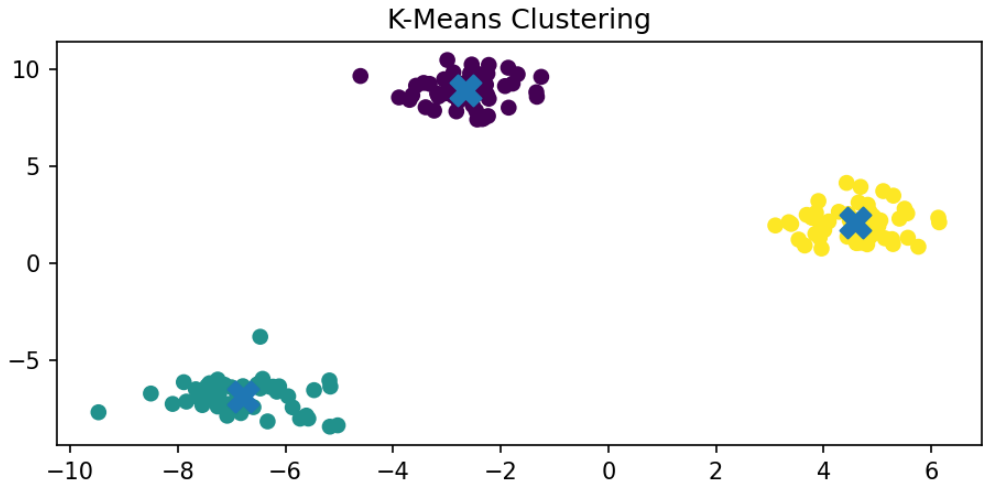
Program / Input Code

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
X, y_true = make_blobs(n_samples=150, centers=3, cluster_std=0.8, random_state=42)
model = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = model.fit_predict(X)
print("Cluster centers:\n", model.cluster_centers_.round(2))
print("Inertia:", round(model.inertia_, 2))
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(model.cluster_centers_[:, 0], model.cluster_centers_[:, 1],
marker="X", s=200, label="Centroids")
plt.title("K-Means Clustering (k=3)")
plt.xlabel("Feature 1"); plt.ylabel("Feature 2")
plt.legend(); plt.show()
```

Expected Output

```
The algorithm groups the 150 data points into 3 clusters.
Cluster centers and inertia are printed by the program.
```

Visualization



Result

K-Means successfully groups similar data points into three clusters and identifies their centroids.